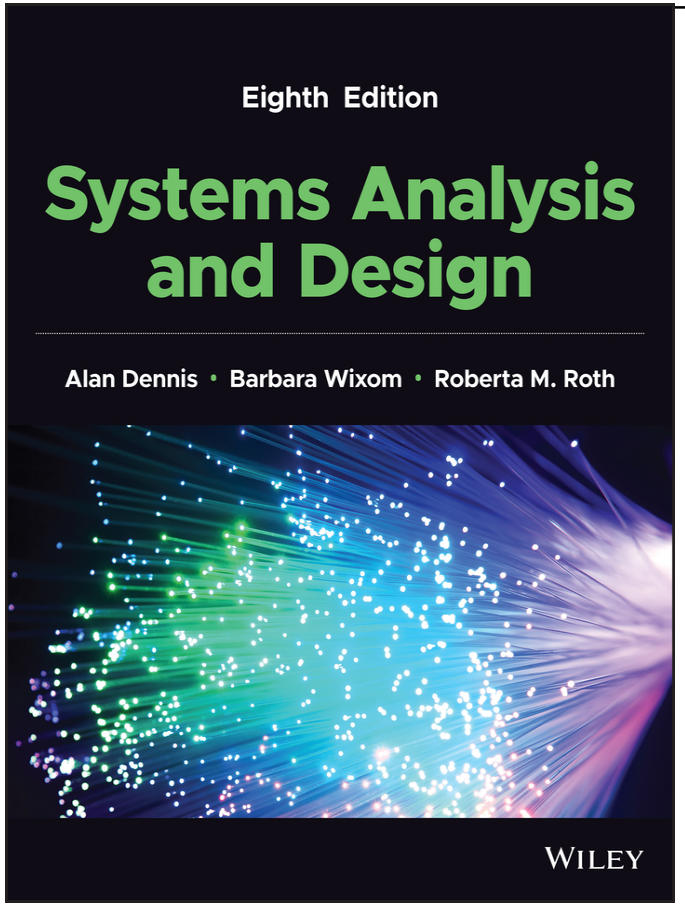
Software Development Lifecycles

Part 3



Systems Analysis and Design

Chapter 9

Program Design

Objectives

- Moving from logical to physical process models.
- Designing programs.
- Structure chart.
- Program specification.

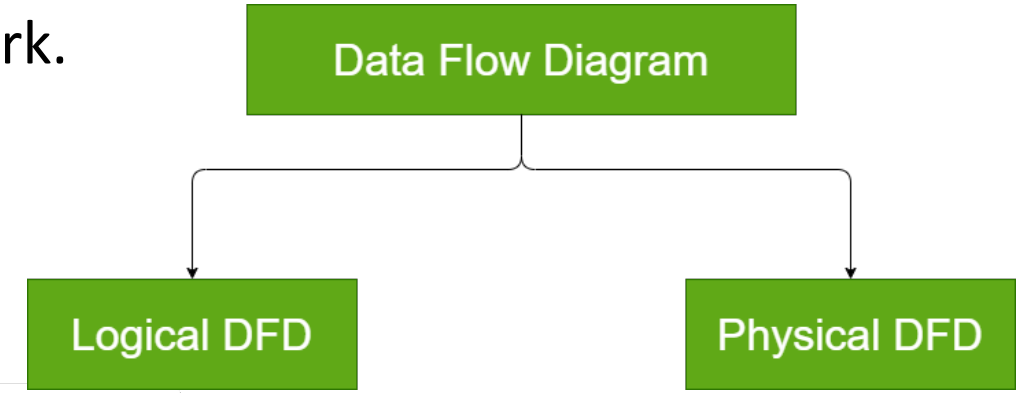
Introduction

- **Program design** – Analysts determine what programs will be written and create instructions for the programmers.
- Various implementation decisions are made about the new system (e.g., what programming language(s) will be used.)
- The DFDs created during analysis are modified to show these implementation decisions, resulting in a set of ***physical data flow diagrams***.
- The analysts determine how the processes are organized, using a ***structure chart*** to depict the decisions.
- Detailed instructions called ***program specifications*** are developed.

BTEC Alliance with FPT Education Pearson

Moving from Logical to Physical Process Models

- The analysis phase focuses on logical data flow diagrams which do not contain any indication how the system will actually be implemented; they simply state what the new system will do.
- During design, ***physical process models*** are created to show implementation details and explain how the final system will work.



The Physical Data Flow Diagram

- The physical DFD contains the same components as the logical DFD, and the same rules apply.
- The basic difference between the two models is that a physical DFD contains additional details that describe how the system will be built.
- There are five steps to perform to make the transition to the physical DFD.

(cont'd)

Step		Explanation
1.	Add implementation references.	Using the existing logical DFD, add the way in which the data stores, data flows, and processes will be implemented to each component.
2.	Draw a human–machine boundary.	Draw a line to separate the automated parts of the system from the manual parts.
3.	Add system-related data stores, data flows, and processes.	Add system-related data stores, data flows, and processes to the model (components that have little to do with the business process).
4.	Update the data elements in the data flows.	Update the data flows to include system-related data elements.
5.	Update the metadata in the CASE repository.	Update the metadata in the CASE repository to include physical characteristics.

FIGURE 9-1 Steps to create the physical data flow diagram.

(cont'd)

- **Step 1: Add Implementation References**
- **Step 2: Draw a Human–Machine Boundary**
- **Step 3: Add System-Related Data Stores, Data Flows, and Processes**
- **Step 4: Update the Data Elements in the Data Flows**
- **Step 5: Update the Metadata in the Computer-Aided Software Engineering Repository**

(cont'd)

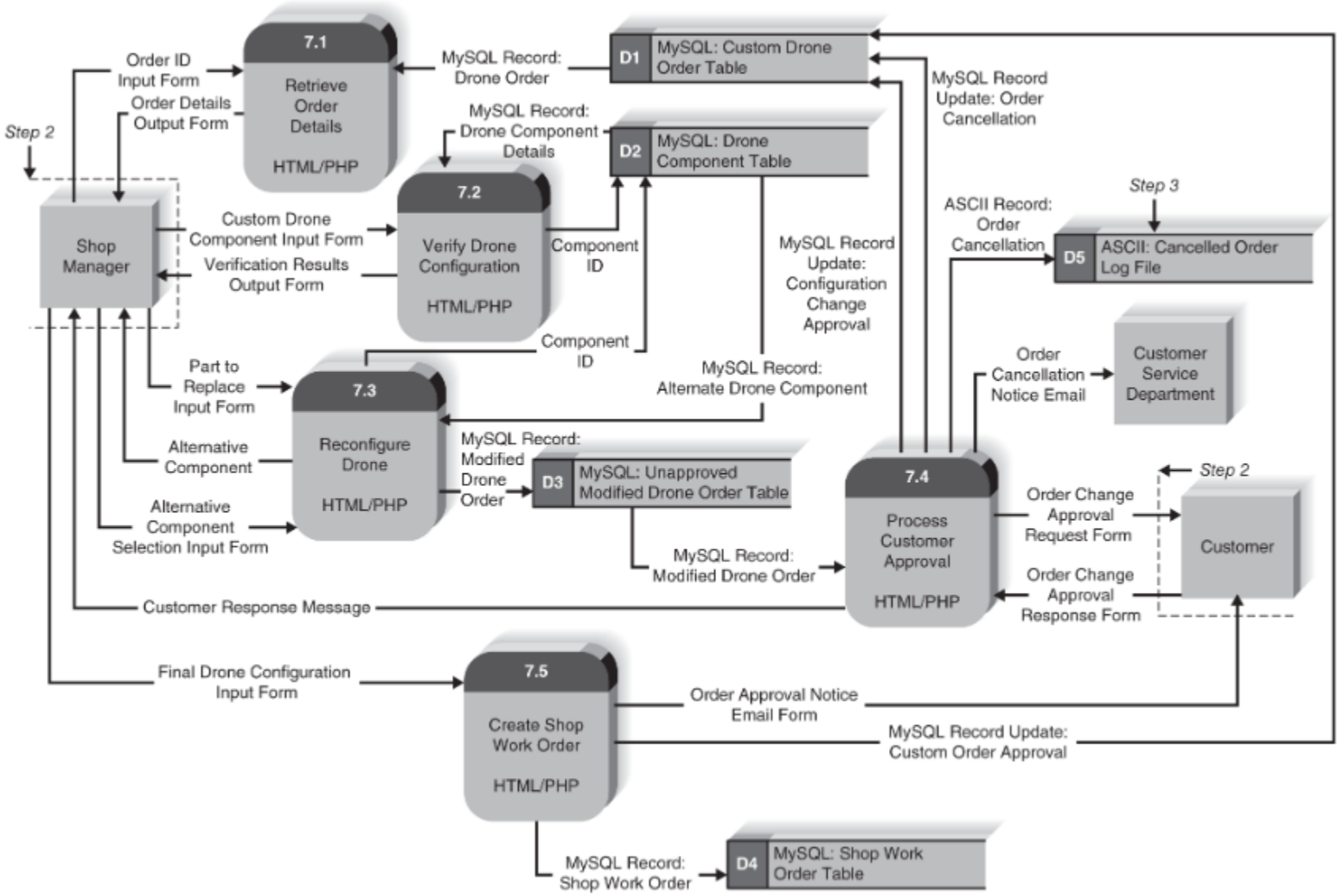


FIGURE 9-2 Physical DFD for shop manager approval of custom drone order level 1 DFD.

(cont'd)

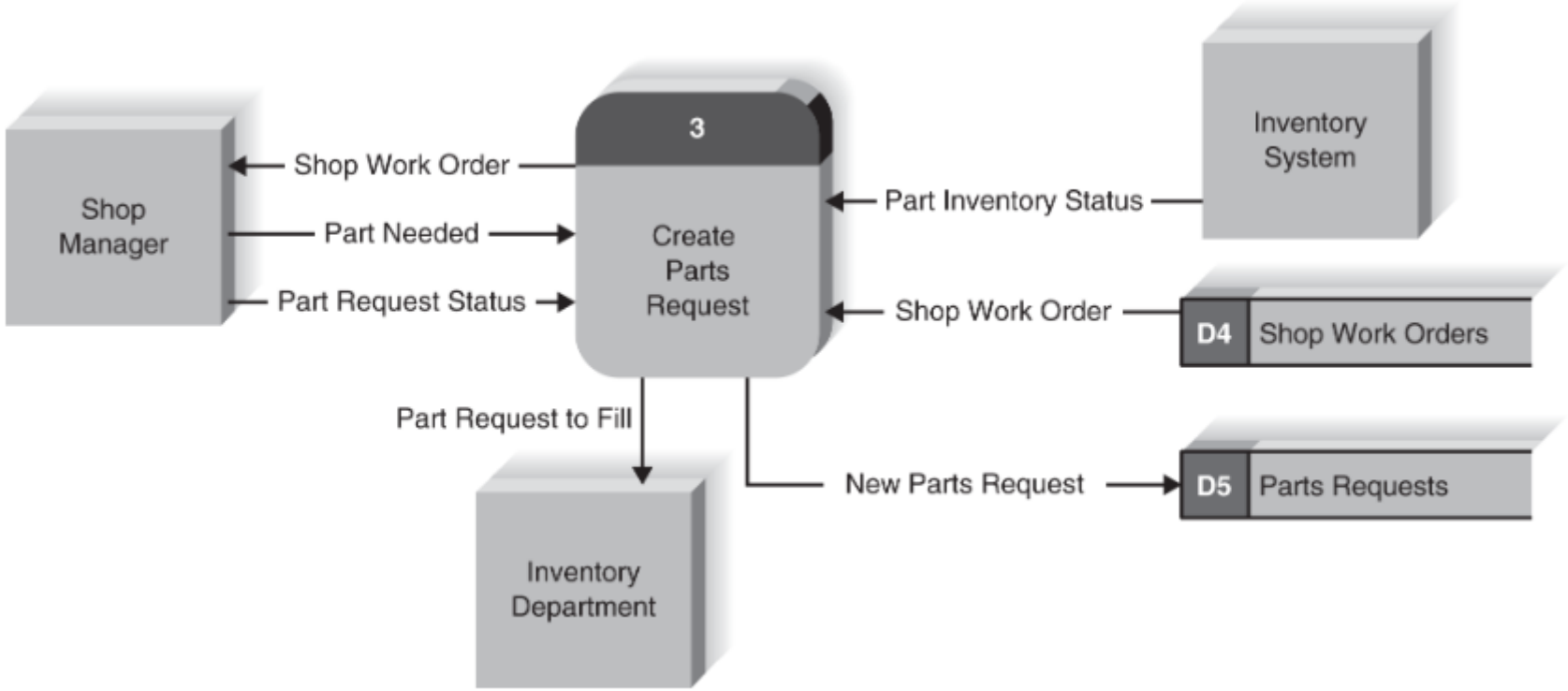


FIGURE 9-3 Logical model of DrōnTeq's shop management process 3: create parts request.

(cont'd)

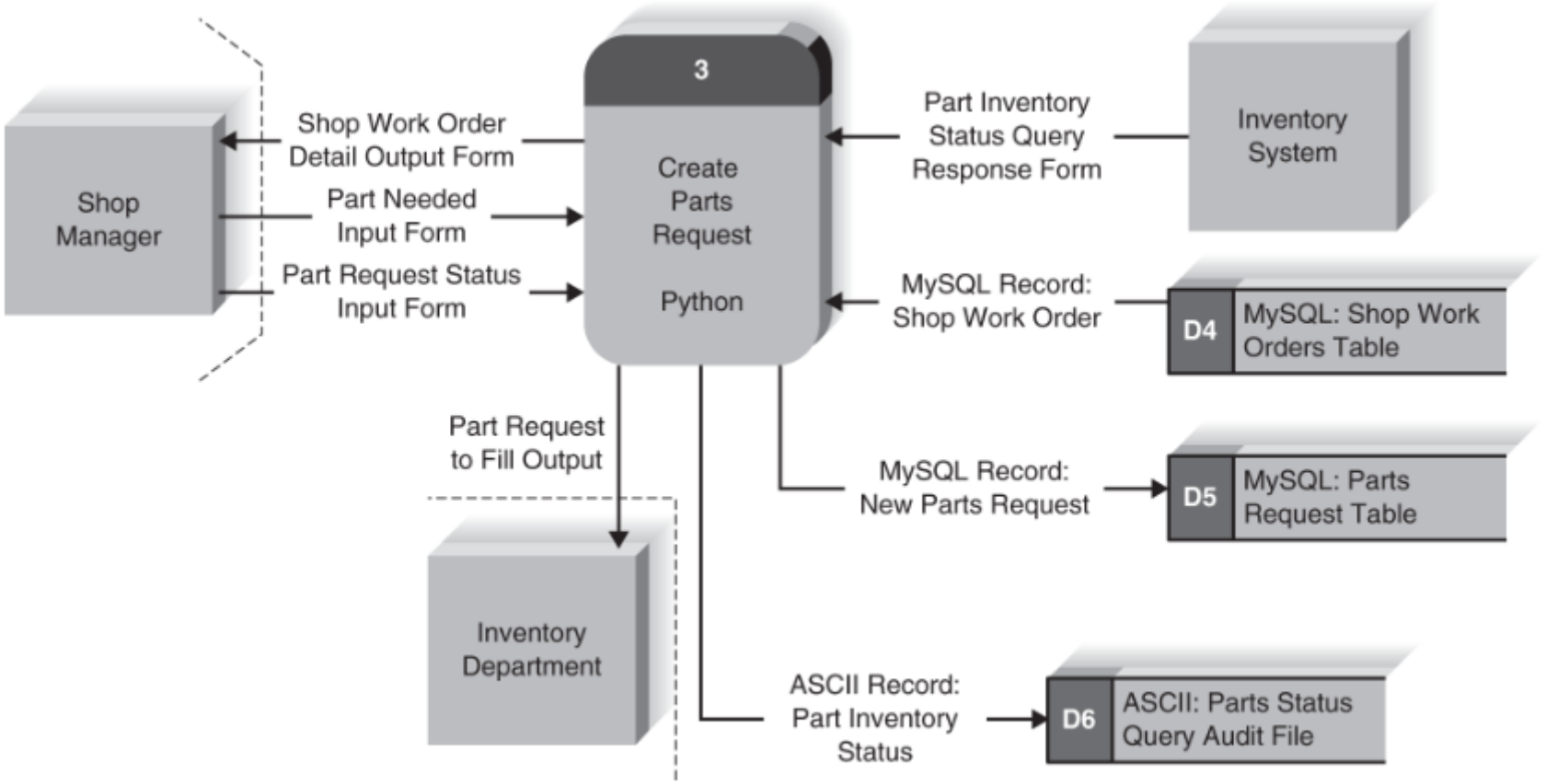
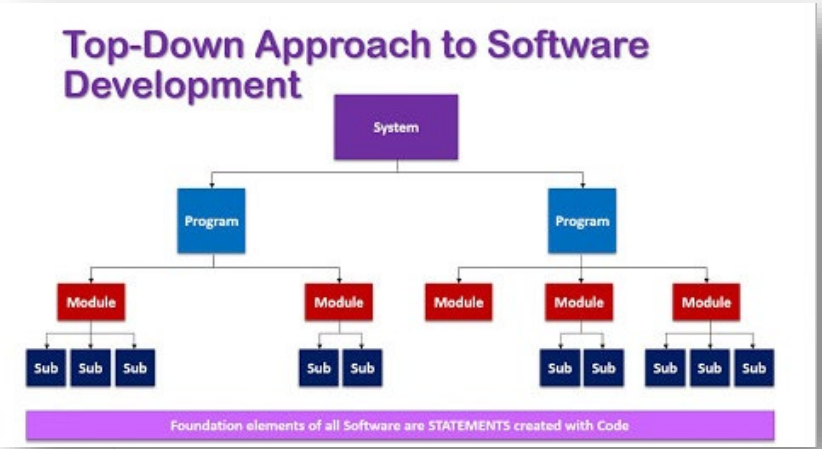


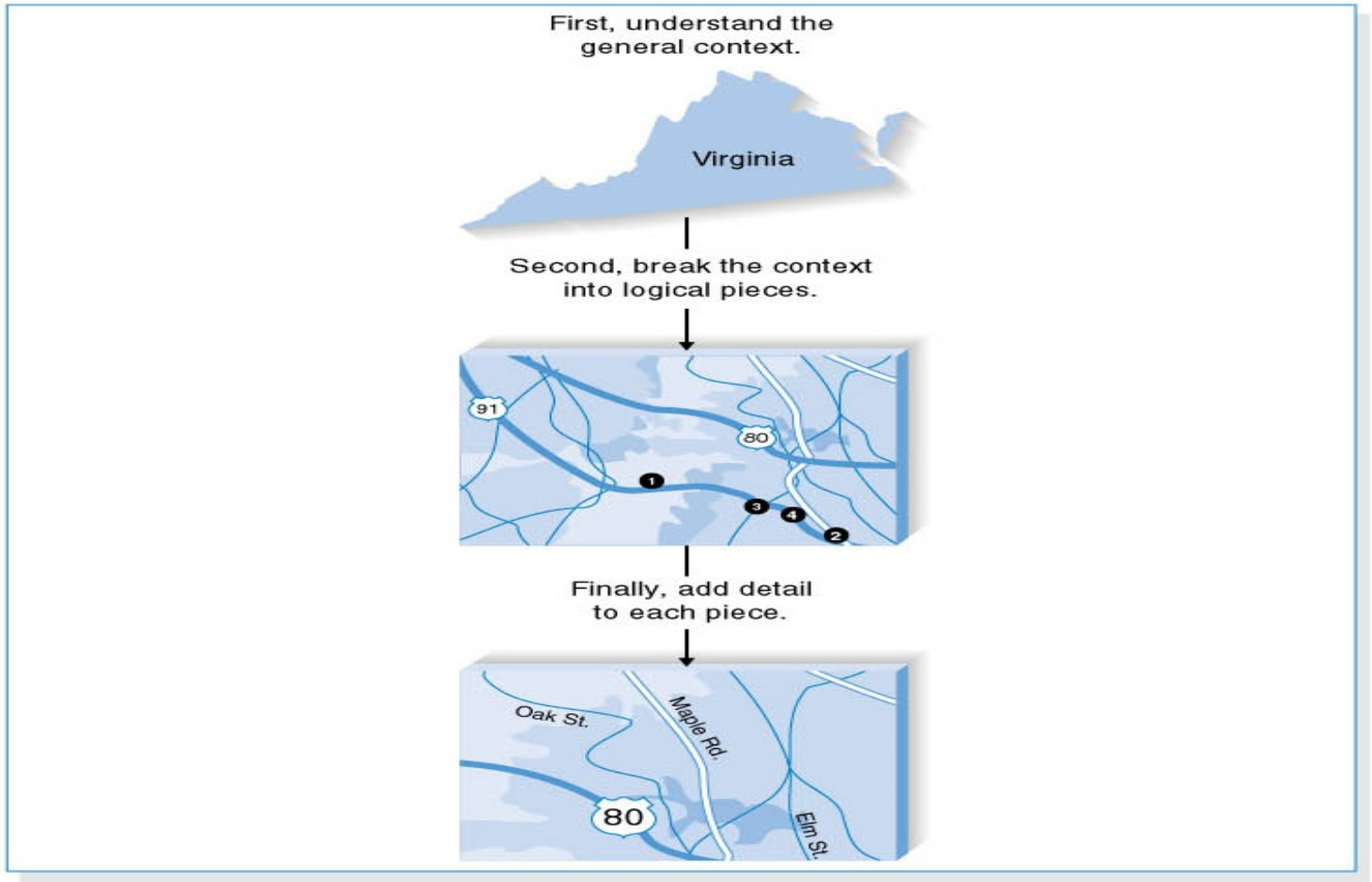
FIGURE 9-4 Physical model for DrōnTeq's shop management process 3: create parts request.

Designing programs

- Analysts should create a design of a maintainable system that is modular and flexible.
- Analysts can design programs in a ***top-down modular approach***.



(cont'd)


First, understand the general context.
Virginia
Second, break the context into logical pieces.
91
80
1
3
4
2
Finally, add detail to each piece.
Oak St.
Maple Rd.
80
Elm St.

(cont'd)

- Program design in the top-down modular approach:

- A high-level diagram, called ***structure chart***, is created to illustrate the organization and interaction of the different pieces of code within the program.

- Program specifications are written to describe what needs to be included in each program module.

- At the end of program design, the project team compiles the ***program design*** document.

Structure Chart

- The ***structure chart*** is an Important program design technique that help the analyst design the program.
- It shows all components of code in a hierarchical format that implies
  - ***sequence***,
  - ***selection***, and
  - ***iteration***.

(cont'd)

Structure Chart Example

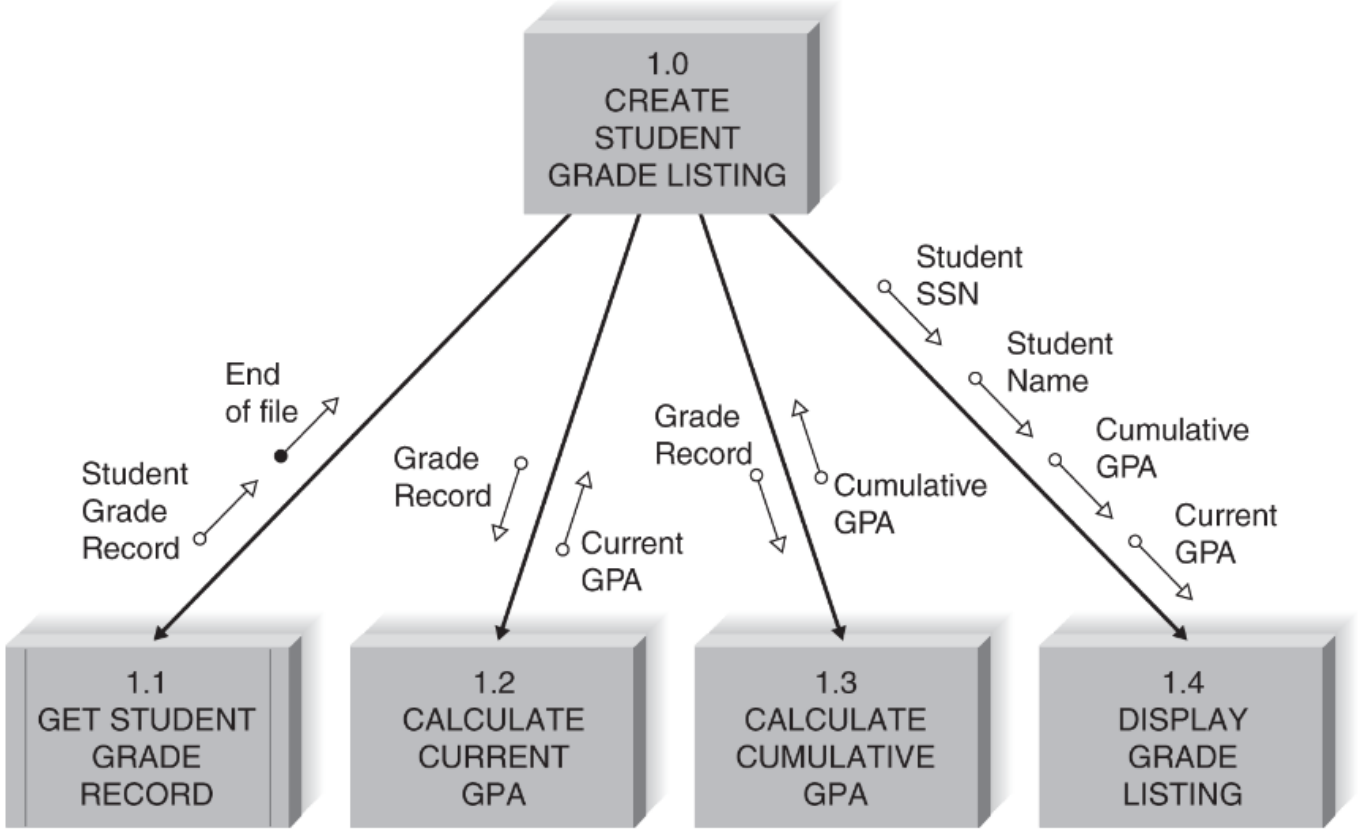
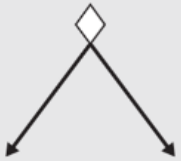


FIGURE 9-6 Structure chart example (GPA = grade point average).

Syntax of Structure Chart

Structure Chart Element	Purpose	Symbol
Every *module*: • Has a number. • Has a name. • Is a control module if it calls other modules below it. • Is a subordinate module if it is controlled by a module at a higher level.	Denotes a logical piece of the program	1.2 CALCULATE CURRENT GPA
Every *library module* has: • A number. • A name. • Multiple instances within a diagram.	Denotes a logical piece of the program that is repeated within the structure chart	1.1 GET STUDENT GRADE RECORD
A *loop*: • Is drawn with a curved arrow. • Is placed around lines of one or more modules that are repeated.	Communicates that a module(s) is repeated	
A *conditional line*: • Is drawn with a diamond. • includes modules that are invoked on the basis of some condition.	Communicates that subordinate modules are invoked by the control module based on some condition	

(cont'd)

A *data couple*: • Contains an arrow. • Contains an empty circle. • Names the type of data that are being passed. • Can be passed up or down. • Has a direction that is denoted by the arrow.	Communicates that data are being passed from one module to another	grade record
A *control couple*: • Contains an arrow. • Contains a filled-in circle. • Names the message or flag that is being passed. • Should be passed up, not down. • Has a direction that is denoted by the arrow.	Communicates that a message or a system flag is being passed from one module to another	end of file
An *off-page connector*: • Is denoted by the hexagon. • Has a title. • Is used when the diagram is too large to fit everything on the same page.	Identifies when parts of the diagram are continued on another page of the structure chart	PRINT GRADE LISTING
An *on-page connector*: • Is denoted by the circle. • Has a title. • Is used when the diagram is too large to fit everything in the same spot on a page.	Identifies when parts of the diagram are continued somewhere else on the same page of the structure chart	PRINT GRADE LISTING

(cont'd)

Revised structure chart example

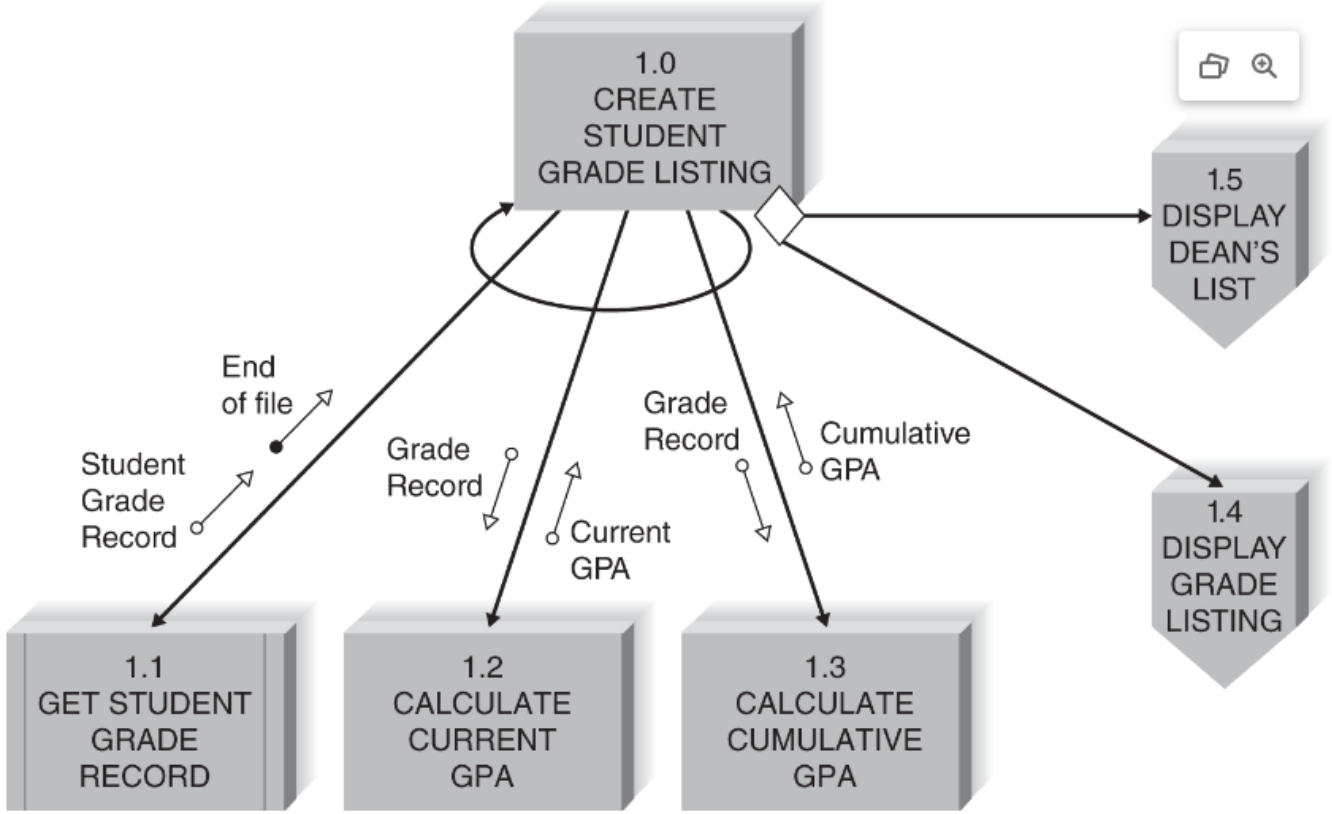


FIGURE 9-8 Revised structure chart example.

Building the Structure Chart

- Process models are used as the start point for structure charts.
- There are three basic kinds of processes:
  - ***Afferent processes*** provide inputs to system,
  - ***Central processes*** perform critical functions in the operation, and
  - ***Efferent processes*** deal with system outputs.
- Each DFD level tends to correspond to a different level of the structure chart hierarchy.

(cont'd)

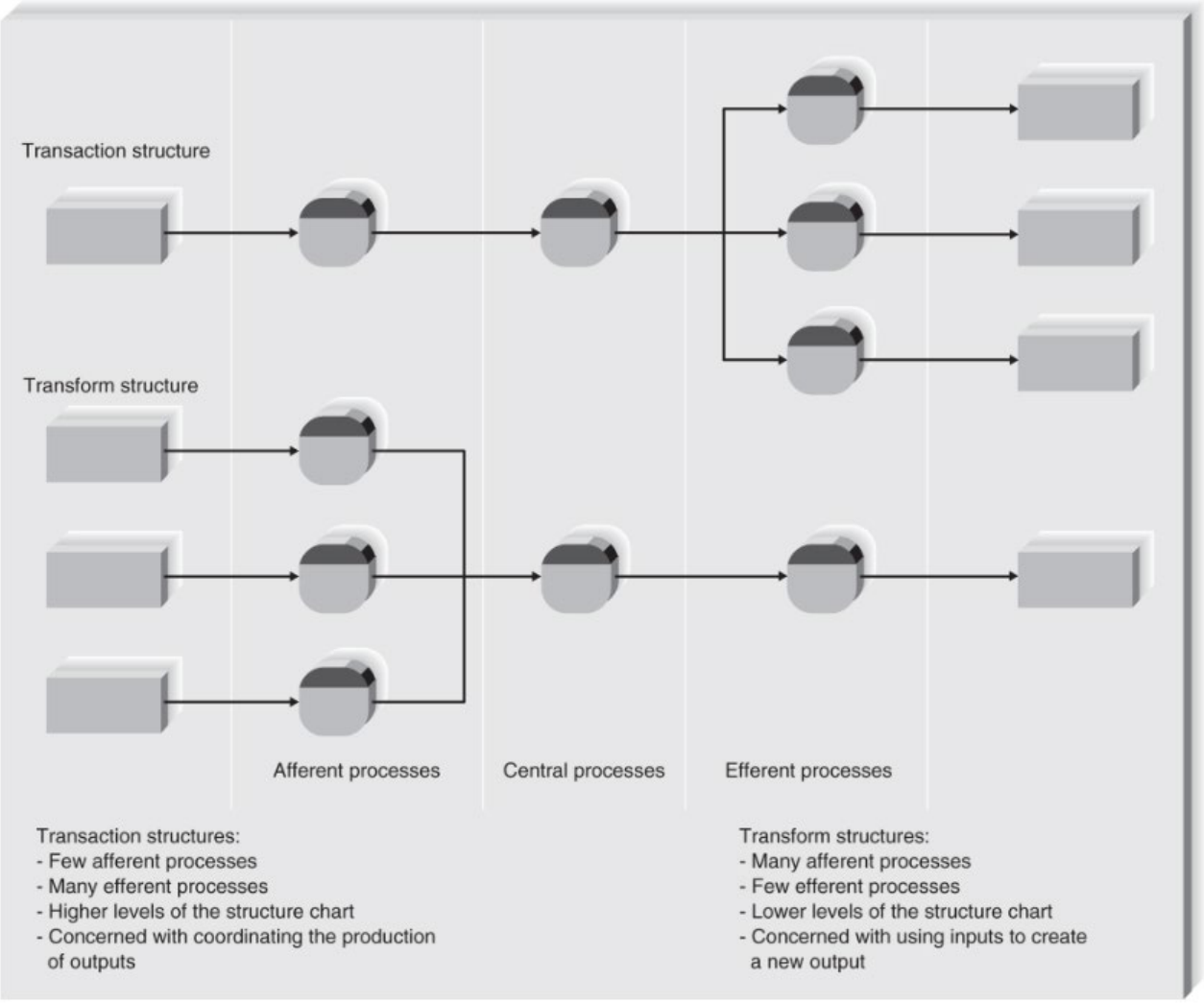


FIGURE 9-9 Transform and transaction structures.

Transaction Structure

- There are two basic arrangement, or structures, for combining modules.
- The ***transaction structure*** contains a control module that calls subordinate modules, each of which handles a particular transaction.
  - Few afferent processes
  - Many efferent processes
  - Higher levels of the structure chart
  - Connected with coordinating the production of outputs

(cont'd)

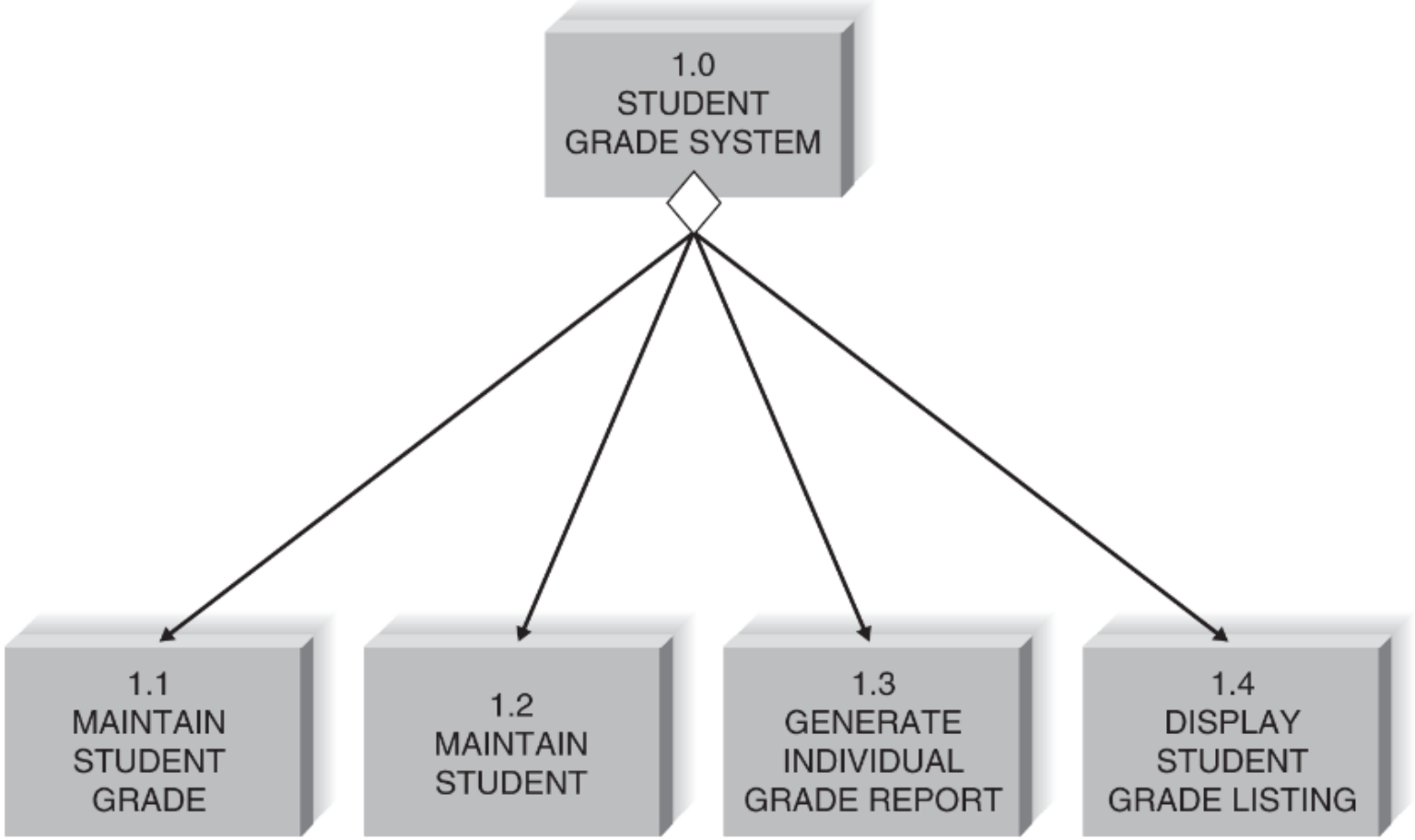


FIGURE 9-10 Transaction structure.

Transform Structure

- The ***transaction structure*** has a control module that calls several subordinate modules in sequence after which something "happens."
- These modules are related because together they form a process that transforms some input into an output.
  - Many afferent processes
  - Few efferent processes
  - Lower levels of the structure chart
  - Connected with using inputs to create a new output

Steps in Building the Structure Chart

1. Identify modules and levels.
2. Identify special connections.
3. Add couples.
4. Revise structure chart.

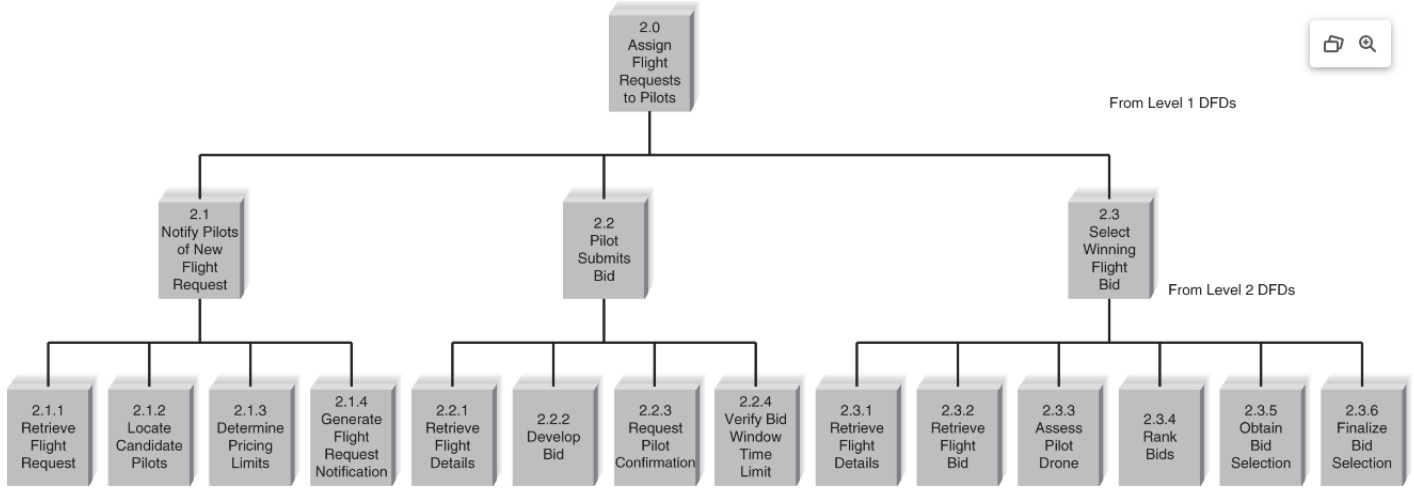


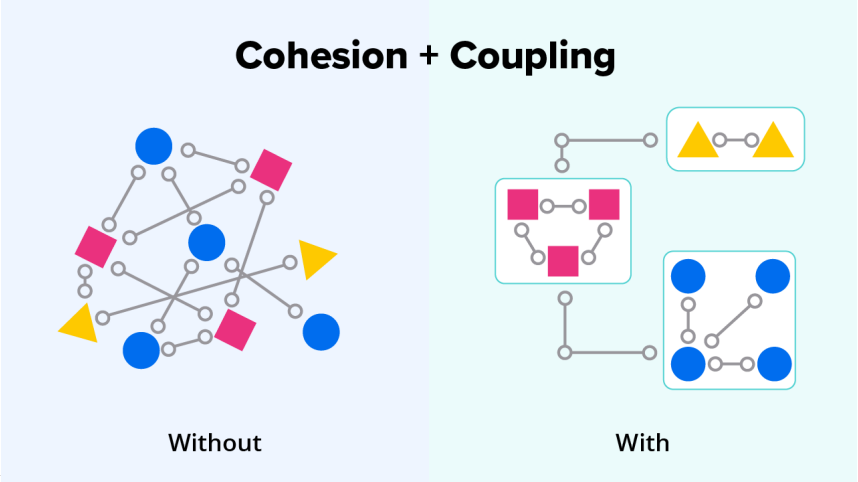
FIGURE 9-11 Step 1: Identify modules and levels for the structure chart.

Design Guidelines

- High quality structure charts result in programs that are modular, reusable, and easy to implement.
- Measures of good design include
  - cohesion,
  - coupling, and
  - appropriate levels of fan-in and fan-out.

Build Modules with High Cohesion

- ***Cohesion*** refers to how well the lines of code within each module relate to each other.
- Cohesive modules are easy to understand and build because their code performs one function effectively.



(cont'd)

- There are various types of cohesion.
- ***Functional cohesion*** – all elements of the modules contribute to performing a single task.
- ***Temporal cohesion*** – functions are invoked at the same time.
- ***Coincidental cohesion*** – there is no apparent relationship among a module's functions.

(cont'd)

	Type	Definition	Example
Good	Functional	Module performs one problem-related task.	Calculate Current GPA The module calculates current GPA only.
	Sequential	Output from one task is used by the next.	Format and Validate Current GPA Two tasks are performed, and the formatted GPA from the first task is the input for the second task.
	Communicational	Elements contribute to activities that use the same inputs or outputs.	Calculate Current and Cumulative GPA Two tasks are performed because they both use the student grade record as input.
	Procedural	Elements are performed in sequence but do not share data.	Print Grade Listing The module includes the following: housekeeping, produce report.
	Temporal	Activities are related in time.	Initialize Program Variables Although the tasks occur at the same time, each task is unrelated.
	Logical	List of activities; which one to perform is chosen outside of module.	Perform Customer Transaction This module will open a checking account, open a savings account, or calculate a loan, depending on the message that is sent by its control module.
	Coincidental	No apparent relationship.	Perform Activities This module performs different functions that have nothing to do with each other: update customer record, calculate loan payment, print exception report, analyze competitor pricing structure.
Bad			

FIGURE 9-14 Types of cohesion (GPA = grade-point average).

(cont'd)

- Cohesion Decision Tree

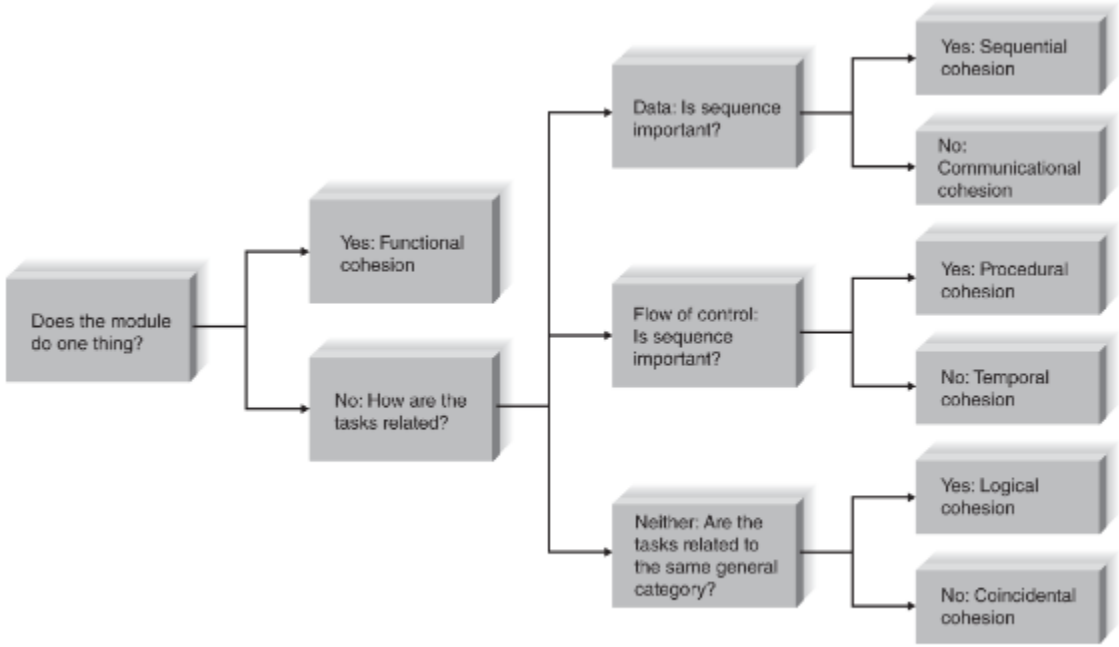


FIGURE 9-15 Cohesion decision tree (adapted from Page-Jones, 1980).

(cont'd)

- ***Factoring*** is the process of separating out a function from one module into a module of its own.
- Factoring can make modules cohesive create a better structure.

Build Loosely Coupled Modules

- ***Coupling*** involves how closely modules are interrelated.
- Modules should be loosely coupled.
- ***Data coupling*** occurs when modules pass parameters or specific data to each other. It is preferred.
- ***Content coupling*** occurs when one module actually refers to the inside of another module. It is a bad coupling type.

Create High Fan-In

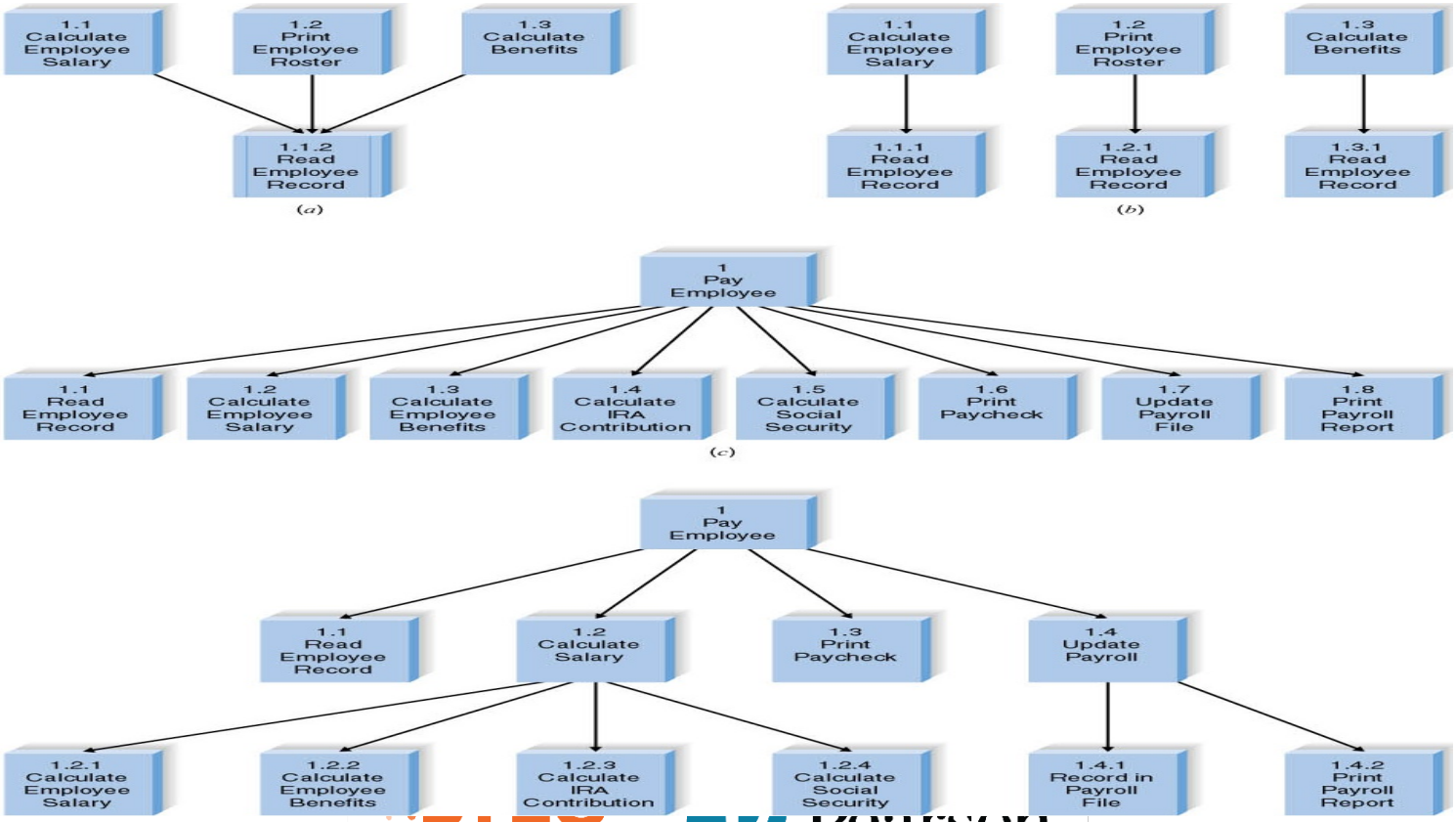
- ***Fan-in*** describes the number of control modules that communicate with a subordinate.
- A module with high fan-in has many different control modules that call it. This is a good situation because high fan-in indicates that a module is reused in many places on the structure chart.

Avoid High Fan-Out

- A large number of subordinates associated with a single control should be avoided.
- The general rule of thumb is to limit a control module's subordinates to approximately seven for low ***fan-out***.

(cont'd)

- Examples of Fan-in and Fan-out



Assess the Chart for Quality

- Check list for structure chart quality

✔ Library modules have been created whenever possible.
✔ The diagram has a high fan-in structure.
✔ Control modules have no more than seven subordinates.
✔ Each module performs only one function (high cohesion).
✔ Modules sparingly share information (loose coupling).
✔ Data couples that are passed are actually used by the accepting module.
✔ Control couples are passed from “low to high.”
✔ Each module has a reasonable amount of code associated with it.

Program Specification

- ***Program Specifications*** are documents that include explicit instructions on how to program pieces of code.
- There is no formal syntax for program specification.
- Four components are essential for program specification:

  - Program information;

  - Events;

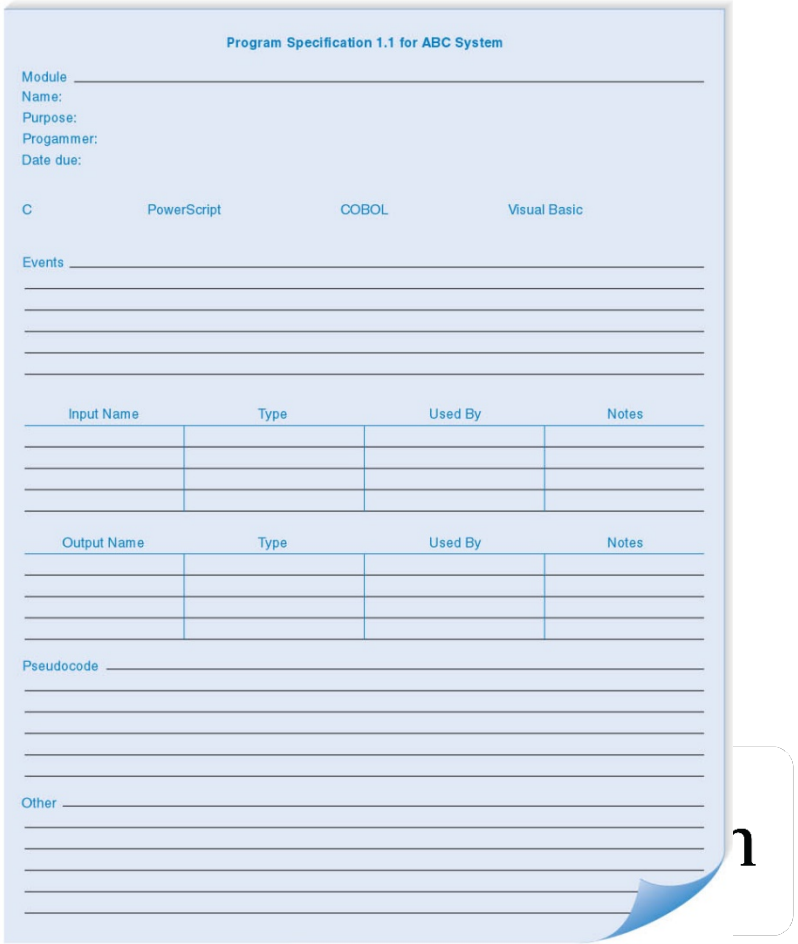
  - Inputs and outputs; and

  - ***Pseudocode*** – a detailed outline of lines of code that need to be written.

- Additional notes and comments can also be included.

(cont'd)

Program specification form



Program Specification 1.1 for ABC System

Module Name:
Purpose:
Progammer:
Date due:

C PowerScript COBOL Visual Basic

Events

Input Name	Type	Used By	Notes

Output Name	Type	Used By	Notes

Pseudocode

Other

(cont'd)

Pseudocode Example

```
(Get_CD_Info module)
    Accept (CD.Title) {Required}
    Accept (CD.Artist) {Required}
    Accept (CD.Category) {Required}
    Accept (CD.Length)
Return
```

SUMMARY

- **Moving from logical to physical process models**.

  - Physical DFDs show implementation details.

- **Structure chart**.

  - The structure chart shows all of the functional components needed in the program at a high level.

- **Building structure chart**.

  - Module, control connection, couples, review.

- **Structure chart design guidelines**.

  - Cohesion, coupling, and fan-in/fan-out.

- **Program specifications**.

  - Program specifications provide more detailed instructions to the programmers.